

REPORTE TÉCNICO DE AUDITORÍA DE SEGURIDAD

PR01 Walkthrough

Prueba de Penetración — Máquina “Snakeoil” (Entorno de Laboratorio Controlado)

Alumno	Manuel Eduardo Hernández Gaytán
Matrícula	181850
Objetivo evaluado	Máquina virtual “Snakeoil” (laboratorio aislado)
Alcance	Red interna host-only, sin conexión a Internet ni sistemas reales
Docente	Mtro. Servando López Contreras
Fecha de realización	9 de septiembre de 2026

Índice

1. Resumen Ejecutivo
2. Metodología y Alcance
3. Fase 1 — Reconocimiento y Enumeración
4. Fase 2 — Análisis de Vulnerabilidades y Explotación
5. Fase 3 — Post-Explotación y Escalada de Privilegios
6. Conclusiones y Recomendaciones de Mitigación

1. Resumen Ejecutivo

El presente documento reporta los resultados de una prueba de penetración de caja gris realizada sobre la máquina virtual “Snakeoil”, desplegada exclusivamente dentro de una red interna de laboratorio (host-only), sin conexión a Internet ni interacción con sistemas o usuarios reales. El ejercicio se realizó con fines exclusivamente académicos, dentro del marco de la asignatura de Seguridad en Tecnologías de la Información.

El objetivo fue evaluar la superficie de ataque de una aplicación web expuesta en el objetivo, siguiendo una metodología estructurada en cuatro fases: reconocimiento, enumeración, explotación y post-explotación. Como resultado del ejercicio, se logró comprometer la aplicación web mediante la explotación de vulnerabilidades de diseño (credenciales embebidas en el código fuente y un endpoint con validación de entradas insuficiente), obteniendo inicialmente ejecución remota de comandos y, posteriormente, acceso con privilegios de administrador (root) sobre el sistema operativo del objetivo.

Los hallazgos evidencian fallas comunes en el ciclo de vida de desarrollo seguro: manejo inadecuado de secretos, validación de entradas basada en listas negras y reutilización de credenciales entre distintos componentes del sistema. Las recomendaciones correspondientes se detallan en la sección de conclusiones.

Resumen de hallazgos

#	Hallazgo	Severidad
1	Credenciales y llaves sensibles embebidas en el código fuente de la aplicación	Alta
2	Endpoint administrativo vulnerable a inyección de comandos del sistema operativo (validación por lista negra evadible)	Crítica
3	Reutilización de contraseñas entre la aplicación y el sistema operativo, facilitando la escalada de privilegios	Alta

2. Metodología y Alcance

El ejercicio se estructuró siguiendo una metodología de pentesting por fases, consistente con marcos de referencia estándar de la industria (p. ej. PTES), adaptada al contexto de un laboratorio académico controlado.

Fase	Objetivo	Herramientas principales
Reconocimiento	Identificar el host objetivo dentro de la red del laboratorio.	netdiscover, nmap
Enumeración	Descubrir servicios, directorios y endpoints expuestos.	gobuster, Burp Suite

Fase	Objetivo	Herramientas principales
Explotación	Validar y explotar las vulnerabilidades identificadas.	curl / cliente HTTP, Burp Suite
Post-explotación	Consolidar el acceso y evaluar la escalada de privilegios.	shell remota, netcat

Alcance y restricciones éticas

- El objetivo evaluado fue exclusivamente la máquina virtual “Snakeoil”, sin extender el análisis a ningún otro sistema.
- Toda la actividad se ejecutó dentro de una red interna host-only, sin exposición a Internet público.
- No se utilizaron credenciales, datos ni sistemas de producción reales en ningún momento del ejercicio.
- No se emplearon técnicas destructivas; el objetivo del ejercicio fue únicamente demostrar el impacto potencial de las vulnerabilidades identificadas.

3. Fase 1 — Reconocimiento y Enumeración

Se inició con la identificación del host objetivo dentro del segmento de red del laboratorio mediante descubrimiento pasivo de tráfico ARP, seguido de un escaneo de puertos y servicios para determinar la superficie de ataque disponible.

```
$ sudo netdiscover -i eth0
$ nmap -sV -p- <IP_objetivo>
```

El escaneo reveló un servicio web expuesto en un puerto no estándar. Sobre dicho servicio se ejecutó una enumeración de directorios y rutas, complementada con el análisis del tráfico HTTP mediante un proxy de interceptación, lo que permitió mapear una API REST interna con un esquema de autenticación basado en JSON Web Tokens (JWT).

```
$ gobuster dir -u http://<IP_objetivo>:<puerto> -w <wordlist>
```

Servicios y hallazgos de la fase

Elemento	Observación
Servicio web	Aplicación backend con API REST, autenticación basada en JWT
Endpoints identificados	Rutas de registro, inicio de sesión, gestión de usuarios y un endpoint administrativo
Mecanismo de sesión	Tokens de acceso y refresco (JWT) transmitidos mediante cookies

4. Fase 2 — Análisis de Vulnerabilidades y Explotación

El análisis del comportamiento de la aplicación y de su código fuente permitió identificar dos vulnerabilidades encadenables que, en conjunto, derivaron en ejecución remota de comandos sobre el servidor.

Vulnerabilidad 1: Credenciales embebidas en el código fuente

Se identificaron valores sensibles (llaves de firma y secretos de aplicación) definidos directamente como constantes dentro del código fuente de la aplicación, en lugar de gestionarse mediante variables de entorno o un mecanismo de gestión de secretos. Esta práctica permite que cualquier persona con acceso al código — o que logre exfiltrarlo— obtenga dichos valores sin necesidad de comprometer credenciales adicionales.

Vulnerabilidad 2: Inyección de comandos del sistema operativo

Se identificó un endpoint administrativo que aceptaba una URL como parámetro y ejecutaba una operación de red utilizando dicho valor. La validación de entradas implementada se basaba en una lista negra de palabras clave, un enfoque conocido por ser evadible mediante técnicas de ofuscación o el uso de utilidades alternativas no contempladas en la lista. Esto permitió inyectar comandos adicionales que el servidor ejecutó con los privilegios del proceso de la aplicación, resultando en la descarga y ejecución de un script que estableció una conexión remota (shell) hacia un equipo bajo control del evaluador.

Resumen de vulnerabilidades explotadas

Vulnerabilidad	Clasificación (CWE)	Impacto
Secretos embebidos en código fuente	CWE-798: Use of Hard-coded Credentials	Exposición de llaves de firma de sesión
Inyección de comandos vía validación por lista negra	CWE-78: OS Command Injection	Ejecución remota de comandos (RCE)

Como resultado de esta fase se obtuvo una shell interactiva sobre el sistema operativo del objetivo, con los privilegios del usuario bajo el cual se ejecutaba la aplicación web.

5. Fase 3 — Post-Explotación y Escalada de Privilegios

Tras obtener acceso remoto inicial, se procedió a estabilizar la sesión mediante la asignación de una terminal interactiva completa, y a explorar el sistema de archivos local en busca de información que permitiera ampliar el nivel de acceso obtenido.

La revisión del entorno del usuario comprometido reveló el código fuente completo de la aplicación, incluyendo una contraseña que resultó estar reutilizada como credencial de dicho usuario a nivel de sistema operativo. Al consultar los permisos de ejecución elevada configurados para ese usuario, se determinó que contaba con permisos suficientes para obtener una sesión con privilegios de administrador.

Hallazgos de la fase

Elemento	Observación
Reutilización de credenciales	La contraseña embebida en el código fuente de la aplicación coincidía con la credencial del usuario del sistema
Configuración de privilegios elevados	El usuario comprometido contaba con permisos suficientes para escalar a administrador
Resultado final	Acceso confirmado con privilegios de administrador (root) sobre el sistema objetivo

El ejercicio concluyó con la confirmación de compromiso total del objetivo dentro del entorno de laboratorio, validando el impacto máximo alcanzable a partir de las vulnerabilidades identificadas en las fases anteriores.

6. Conclusiones y Recomendaciones de Mitigación

El ejercicio demuestra cómo la combinación de prácticas inseguras de desarrollo —manejo inadecuado de secretos, validación de entradas insuficiente y reutilización de credenciales— puede derivar en el compromiso total de un sistema, incluso sin la presencia de vulnerabilidades de día cero o técnicas sofisticadas. Cada hallazgo, evaluado de forma aislada, ya representa un riesgo relevante; su combinación amplificó significativamente el impacto final.

Recomendaciones de mitigación

#	Recomendación	Prioridad
1	Eliminar cualquier secreto o credencial embebida en el código fuente; gestionar llaves y contraseñas mediante variables de entorno o un gestor de secretos dedicado.	Alta
2	Remover cualquier endpoint de depuración o administrativo que permita ejecución de comandos antes de pasar a producción; si es indispensable, restringir su acceso y aplicar autenticación reforzada.	Crítica
3	Sustituir la validación de entradas basada en listas negras por un enfoque de lista blanca (allowlisting) que defina explícitamente los valores permitidos.	Alta
4	Prohibir la reutilización de	Alta

#	Recomendación	Prioridad
	contraseñas entre distintos servicios, aplicaciones y cuentas del sistema operativo.	
5	Aplicar el principio de mínimo privilegio en la configuración de permisos elevados (sudoers), limitando los comandos que cada usuario puede ejecutar.	Media
6	Incorporar análisis estático de código (SAST) y escaneo de secretos en el pipeline de integración continua.	Media

En conjunto, estas medidas permiten reducir significativamente la probabilidad de que fallas de diseño similares deriven en un compromiso total del sistema, reforzando la postura de seguridad de la aplicación en cada etapa de su ciclo de vida.